Hitesh Rathore(HR) *Notes*

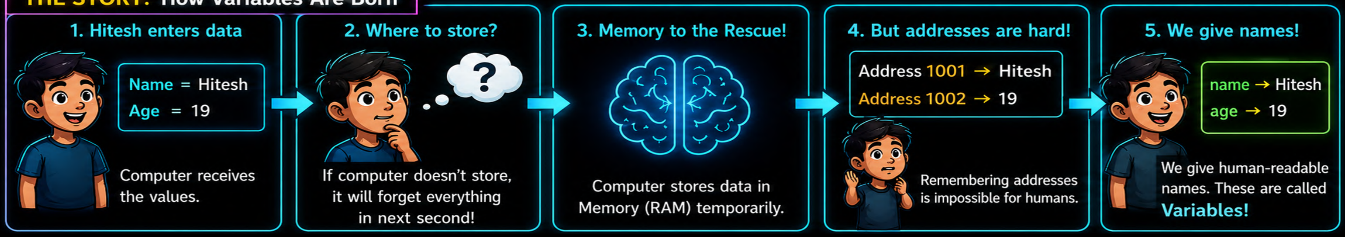
LINKEDIN LINK:
https://www.linkedin.com/in/hitesh-rathore-835279305/

JAVASCRIPT VARIABLES

let vs const vs var

The Complete Visual Story

THE STORY: How Variables Are Born



1. Hitesh enters data

Name = Hitesh
Age = 19

Computer receives the values.

2. Where to store?

If computer doesn't store, it will forget everything in next second!

3. Memory to the Rescue!

Computer stores data in Memory (RAM) temporarily.

4. But addresses are hard!

Address 1001 → Hitesh
Address 1002 → 19

Remembering addresses is impossible for humans.

5. We give names!

name → Hitesh
age → 19

We give human-readable names. These are called **Variables!**

HOW MEMORY LOOKS?

RAM is like a row of thousands of boxes.

☐ · ☐ · ☐ · ☐ · ☐ · ☐ · ☐ ···

Each box can store data.

Hitesh | 19 | true ···

VARIABLE = NAME FOR MEMORY LOCATION

name → Hitesh - - - - - → Address 1001

age → 19 - - - - - → Address 1002

BENEFIT

- ✓ No need to remember addresses
- ✓ Just remember names

CREATING VARIABLES IN JAVASCRIPT

```
let age = 19;
```

VISUAL REPRESENTATION

age
19

What happens?

1. Memory is created
2. Name age is given
3. Value 19 is stored

LET – Value Can Change

Before
age
19

```
age = 20;
```

After
age
20

let → value can change in future

REAL LIFE EXAMPLE (LET)

Bank Balance

```
let balance = 1000;
```

```
balance = 5000;
```
(Salary)

```
balance = 3000;
```
(Shopping)

Balance changes → let is perfect!

WHY CONST?

Not every value should change.

Example: PI = 3.14159

If someone changes it by mistake:

```
PI = 10;
```

Whole program can break!

So we lock some variables.

CONST – Value Cannot Change (Locked)

```
const PI = 3.14159;
```

```
PI = 10;
```
✗ Error!

PI
3.14159
Locked

Declaration and assignment must be together.
You must declare and assign a value at the same time.
Otherwise, you will get an error.
Example of error: `const PI;`
✗ SyntaxError: Missing initializer in const declaration

REAL LIFE EXAMPLES (CONST)

```
const country = "India";
const MAX_USERS = 100;
const PI = 3.14159;
```

These values remain fixed during program execution.

HISTORY OF VAR

In 1995, JavaScript had only one keyword:

var

1995

Example:

```
var age = 19;
```

At that time:
let did not exist
const did not exist
Only var existed

PROBLEMS WITH VAR ⚠

- Scope issues
- Hoisting confusion
- Redeclaration problems

Example:

```
var age = 19;
var age = 20;
```
✓ Allowed

Allowed, but can create bugs in large projects.

ES6 SOLUTION (2015)

In 2015, ECMAScript 2015 (ES6) came.

Introduced:

let **const**

To solve old problems and make JavaScript better.

MODERN RULE

Value change hogi?

Yes ↓ **let** — Use let

No ↓ **const** — Use const

var ? Mostly avoid. ⊗

FINAL MENTAL MODEL

- Computer stores data
↓
- Data is stored in memory
↓
- Memory addresses are assigned
↓
- We create names (identifiers)
↓
- These names are called **Variables**
↓
- In JavaScript, we create variables using:

let, const, var

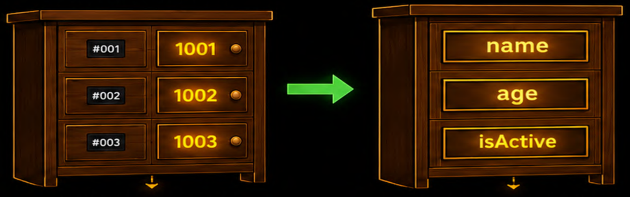
let
Value can change

const
Value cannot change

var
Old way, generally avoid

REAL WORLD ANALOGY

VARIABLE = LABELED BOX



Without Labels (Remember Address)

Hard to remember!

With Labels (Variables)

Easy to remember!

MORE ANALOGIES

Variable = Labeled Box
Memory = Big Warehouse
let = Box whose value can change
const = Locked Box (value cannot change)
var = Legacy System (outdated)

PRO TIP

Always prefer `let` and `const`. They make your code clean, predictable, and bug-free!

</> SEMICOLONS IN JAVASCRIPT </>

1. ARE SEMICOLONS MANDATORY?

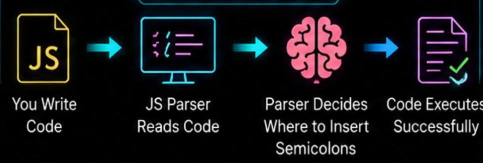
No! In JavaScript, semicolons (;) at the end of each line are **NOT** mandatory.

JavaScript has a powerful feature:

Automatic Semicolon Insertion (ASI)

The JavaScript parser automatically inserts semicolons in some situations.

WHAT THIS MEANS



You Write Code → JS Parser Reads Code → Parser Decides Where to Insert Semicolons → Code Executes Successfully

Key Idea: The parser tries to guess where one statement ends and the next one begins.

2. HOW IT WORKS (EXAMPLE)

You Write

```
let name = "Hitesh"
console.log(name)
```

↓

JavaScript Treats Internally

```
let name = "Hitesh";
console.log(name);
```

Why it works?

ASI adds semicolons automatically, so the code runs fine.

FIRST PRINCIPLES

The parser needs to know:

Where does statement 1 end? → Where does statement 2 start?

With Semicolons (Clear)	Without Semicolons (Guess)
`let a = 10;` `let b = 20;`	`let a = 10` `let b = 20`
✓ No ambiguity. Parser is 100% sure. ✓	✗ Parser has to guess. Sometimes it guesses wrong. ✗

⚠ WHERE PROBLEMS HAPPEN

In some cases, JavaScript inserts semicolons in places you don't expect.

EXAMPLE 1: RETURN PROBLEM

YOU WRITE (EXPECTATION)

```
function getData() {
    return
    {
        name: "Hitesh"
    }
}
```

✓ You expect to return an object. 🤔

WHAT JAVASCRIPT DOES (ACTUAL)

```
function getData() {
    return;
    {
        name: "Hitesh"
    }
}
```

← ASI inserts semicolon after return

Result: undefined ✗

✗ Object is not returned!

INDUSTRY APPROACHES

① ALWAYS USE SEMICOLONS

```
let a = 10;
let b = 20;
console.log(a);
```

- ✓ Safe
- ✓ Explicit
- ✓ Avoids ambiguity
- ✓ Good for beginners

② NEVER USE SEMICOLONS

```
let a = 10
let b = 20
console.log(a)
```

- ✓ Cleaner look
- ! Need to understand ASI rules

③ LET FORMATTER HANDLE

Teams usually use tools like Prettier, ESLint to decide.

Prettier + ESLint

★ Most companies follow a consistent style across the entire codebase.

BEST PRACTICE (FOR BEGINNERS)

Use semicolons at the end of every statement.

```
let name = "Hitesh";
let age  = 19;

console.log(name);
console.log(age);
```

Why?

- ✓ Safe
- ✓ Predictable
- ✓ Readable
- ✓ Beginner Friendly
- ✓ Avoids ASI issues

JS DATA TYPES IN JAVASCRIPT JS

1. REAL PROBLEM

CASE 1: NUMBERS

```
let a = 10;
let b = 20;
a + b
```

↓

`30`

(Addition)

CASE 2: STRINGS

```
let a = "10";
let b = "20";
a + b
```

↓

`"1020"`

(Concatenation)

?

Both cases store some bits in memory.

Question:

How does JavaScript know what to do in each case?

Answer:

Data Type ✓

2. MOST IMPORTANT POINT

Data Type is not for the value, it is for the operations.

The engine must know: What can be done with this value?

✓ `true + true`

↓

JavaScript knows:

`true → Boolean`

It applies Boolean rules.

Aa `"hello".toUpperCase()`

↓

JavaScript knows:

`"hello" → String`

It gives String methods.

3 DEFINITION

A Data Type is a label that tells the engine:

- What kind of data it is
- What operations are allowed on it

4. FINAL MENTAL MODEL

Data Type helps JavaScript engine understand:

1. What kind of data this is
2. What operations are valid
3. How to represent it in memory
4. How to interpret it

Example:

```
let age = 19;
```

↓

```
Value  : 19
Type   : Number
```

The type decides how +, -, methods, comparisons, memory representation, everything behaves.

5 ENGINE KNOWS TWO THINGS

For **every value**, JavaScript engine knows:

1 VALUE What the data is

2 TYPE What kind of data it is

Code	Value	Type	Result
`let age = 19;`	19	Number	123
`let name = "Hitesh";`	Hitesh	String	"abc"

✓ Engine always stores **Value** + **Type**

Type decides what **operations** are valid.

Helps in memory representation and correct **interpretation**.

DATA TYPES

PRIMITIVE

- 123 Number
- “ ” String
- ✓ Boolean
- U Undefined
- ∅ Null
- S Symbol
- n̄ BigInt

NON-PRIMITIVE (REFERENCE TYPES)

- { } Object
- [ ] Array
- ƒ() Function
- ••• etc.

JS NUMBER 01

DATA TYPE IN JAVASCRIPT

WHY NUMBER?

Computer needs to know that these values are **mathematical** so it can perform maths on them.

Age	Price	Temperature	Marks
19	99.99	38	95

⇨ TYPE: **NUMBER** ✓

① WHAT IS NUMBER?

JS

In JavaScript, these all are of type **Number.**

Examples:

`19` `99.99` `-500` `0` `•••`

② TYPE CHECK

JS

`typeof 19`	→	`"number"`
`typeof 99.99`	→	`"number"`
`typeof -500`	→	`"number"`

③ INTEGER NUMBERS

Positive Integers

`19` `95` `1000` •••

`typeof → "number"`

Negative Integers

`-10` `-5000` `-1` •••

`typeof → "number"`

④ FLOATING POINT NUMBERS (DECIMALS)

`0.1` `99.99` `3.14159` `6.2` `0.0001` •••

`typeof → "number"`

19 and 19.99 both are of type number.
No int, float, double in JavaScript.

⑤ ZERO

0

`typeof → "number"`

-0

`typeof → "number"`

JavaScript me 0 aur -0 dono exist karte hain.

⑥ SCIENTIFIC NOTATION

Large Numbers

`1e6` `7.9e9` `2e12` •••

(1×10^{6}) (7.9×10^{9})

`typeof → "number"`

Small Numbers

`1e-6` `2.5e-10` `5e-20` •••

(1×10^{-6}) (2.5×10^{-10})

`typeof → "number"`

⑦ HEXADECIMAL

16

`0xff` `0x1A` `0xABC` •••

`typeof → "number"`

⑧ BINARY

101

`0b1010` `0b1111` `0b1000000` •••

`typeof → "number"`

⑨ OCTAL

8

`0o12` `0o77` `0o377` •••

`typeof → "number"`

⑩ INFINITY

Infinity
(1 / 0)

`typeof → "number"`

-Infinity
(-1 / 0)

`typeof → "number"`

⑪ NaN (NOT A NUMBER)

NaN
(0 / 0)

`typeof → "number"`

NaN

JS STRING 02

DATA TYPE IN JAVASCRIPT

① WHY DO WE NEED STRINGS?

Not Everything in the World Is a Number

STEP 1 – WHAT DO WE ALREADY HAVE?

JS

```
let age   = 19;
let marks = 95;
let price = 99.99;
```

All of these are:

✓ Number Data Type

THE BIG QUESTION

Can every piece of information be represented as a **Number**?

EXAMPLE 1	EXAMPLE 2	EXAMPLE 3	EXAMPLE 4
User Name	City	Email	Message
`Hitesh`	`Bangalore`	`hitesh@gmail.com`	`Hello World`
Number?	Number?	Number?	Number?
☒ No	☒ No	☒ No	☒ No

PROBLEM DISCOVERED

Number Data Type
↓
Can only represent mathematical values

☑ Examples (Works)	☒ But it cannot represent
19	Names
95	Cities
99.99	Emails
-500	Messages
...	Text
	...

NEW DATA TYPE INCOMING...
STRING (Text)

CONCLUSION

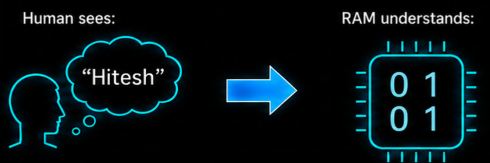
Numbers are not enough.

We need a new data type for
- textual information.

② HOW CAN A COMPUTER STORE TEXT?

Suppose we want to store: `Hitesh`

COMPUTER THINKING



PROBLEM

How can `"Hitesh"` be converted into `0s and 1s ?`

3 SMALLEST BUILDING BLOCK

"Anjana om Kashyap" ko todte hain.

A n j a n a o m K a s h y a p

↓

Word

↓

Built from characters

Bangalore

B a n g a l o r e

Hello

H e l l o

Text
↓
Characters
↓
Smallest Unit

4 HOW CAN WE STORE CHARACTERS?

H → RAM → ?

Character

Memory does not understand by itself

Memory understands only 0 and 1 not character by itself

In the world, we have:

- A-Z, a-z
- 0-9
- Symbols
- Hindi
- Chinese
- Emoji

Give each character a unique number.

A = 1 | B = 2 | C = 3 | • • • | H = 8

So, we need standardized mapping:

ASCII
↓
Unicode

Character → Unique Number → Binary → Memory

5 WHAT IS A STRING?

H → one character
i → one character
t → one character
e → one character
s → one character
h → one character

"Hitesh"
↓
It is a sequence of characters.

And we call this sequence by the name:

STRING

Definition:
String = An ordered sequence of characters.

6 HOW DOES JAVASCRIPT WORK?

If you write:

```
let name = Hitesh;
```

Is it a variable?

Or a text?

Confusing! ☹

So JavaScript has a rule:

Anything written inside quotes is text.

```
let name = "Hitesh";
```

Now JS understands:

✓ This is text.

7 DIFFERENT TYPES OF QUOTES

Double Quotes	Single Quotes	Backticks
"Hitesh"	'Bangalore'	\`Hello\`

We can create three types of strings.

8 DIFFERENT LOOKING STRINGS

Single Character
"A"
This is a string.

Word
"Hitesh"
This is a string.

Multiple Words
"I am learning JavaScript"
This is a string.

Number As Text
"19"
It looks like a number, but it is a string, not a number.

Empty String
" "
This is a string.
Length: 0

Space String
" "
This is not empty.
It has 1 space character.

Special Characters
"!@#$%^&*"
This is a string.

Emojis
"🔥"
This is a string.

Hindi
"नमस्ते"
This is a string.

Mixed
"Hitesh 19 🔥"
This is a string.

```
console.log(typeof "19");
Output: "string"
```

9 MEMORY MENTAL MODEL

```
let name = "Hitesh";
```

name
↓

Index:	0	1	2	3	4	5
	H	i	t	e	s	h

10 WHY DOES LENGTH EXIST?

How many characters are in the string?

```
let name = "Hitesh";
console.log(name.length);
```

Output: 6

H	i	t	e	s	h
1	2	3	4	5	6

So, length = 6

11 WHY IS INDEXING IMPORTANT?

First we choose a character.

```
console.log(name[0]);   →  "H"
console.log(name[1]);   →  "i"
console.log(name[5]);   →  "h"
```

Through indexing, we can access any character at a specific position.

JS

BOOLEAN

DATA TYPE IN JAVASCRIPT

03

</>

1 WHAT IS BOOLEAN?

true VS **false**

★ Only Two Values Exist.
No Third Value.

Real World

User Logged In?
Yes / No

Light Status?
ON / OFF

Exam Result?
Pass / Fail

Think

Boolean represents **two-state** information.

2 TYPE CHECK

`typeof true` → `"boolean"`

`typeof false` → `"boolean"`

Both **true** and **false** are of type **boolean**.

Memory Representation

true		false
1 Bit	VS	1 Bit
(True)		(False)

3 BOOLEAN REPRESENTS CONDITIONS

`10 > 5`
Is 10 greater than 5?
↓
true

`10 < 5`
Is 10 less than 5?
↓
false

★ Boolean represents whether **a condition** is **satisfied.**

Decision Flow

Condition → Boolean → Decision

Every comparison, check or validation returns a **Boolean.**

4 COMPARISONS PRODUCE BOOLEANS

`10 > 5` → true ✓

`10 < 5` → false ✗

`10 == 10` → true ✓

`10 != 10` → false ✗

Comparison
↓
Boolean

5 BOOLEAN IN THE REAL WORLD

Login Successful?
↓
true / false

Password Correct?
↓
true / false

User Is Admin?
↓
true / false

Payment Successful?
↓
true / false

Email Verified?
↓
true / false

File Exists?
↓
true / false

★ Almost every decision in programming is based on **Booleans.**

6 FINAL FIRST-PRINCIPLES MENTAL MODEL

Question → **Condition** → **Evaluation** → **Boolean** → **Decision**

Question	Condition	Evaluation	Boolean	Decision
We ask something.	We create a condition.	JavaScript evaluates it.	Result is either true or false.	Program takes decision.

- ✓ Boolean does not store text.
- ✓ Boolean does not store numbers.
- ✓ Boolean stores the result of a condition.
- ✓ Only two outcomes are possible:

true and **false**

FINAL MEMORY HOOK

QUESTION
↓
ANSWER
↓
BOOLEAN

LOGIC
DECISION

BOOLEAN = THE LANGUAGE OF DECISIONS

{ } EVERY IF, WHILE, FOR, VALIDATION, AUTHENTICATION, COMPARISON → ULTIMATELY RELIES ON **BOOLEAN.**

JS

JS

UNDEFINED

DATA TYPE IN JAVASCRIPT

04

1 THE PROBLEM

So far we discovered:

NUMBER	STRING	BOOLEAN
123	"Hello"	true / false
Mathematical Values	Text / Characters	True or False

Now a new question comes:

Suppose you created an account.

`let account;`

Question:

Account is created. But what is inside it?

The variable exists,
but **no value** has been assigned yet.

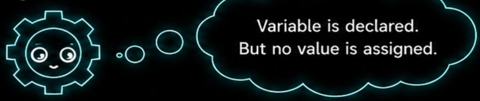
This state is called: **undefined**

2 JAVASCRIPT ENGINE'S PERSPECTIVE

Engine sees the code:

```
let account;
```

Engine thinks:


Variable is declared.
But no value is assigned.

What should the engine keep inside account?

✗	✗	✗	✓
`0 ?`	`"" ?`	`false ?`	Because the user did not give any value.
No.	No.	No.	

Engine's Answer:

undefined → It means: Variable exists,
but **value is not assigned** yet.

3 JAVASCRIPT KA SOLUTION

The variable exists.
But currently it has
no assigned value.

↓

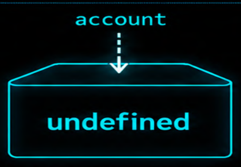
This state is called:

undefined

4 MEMORY VISUALIZATION

```
let  account;
```

Conceptually:



5 LIVE PREVIEW

CODE

```
let account;
console.log(account);
```

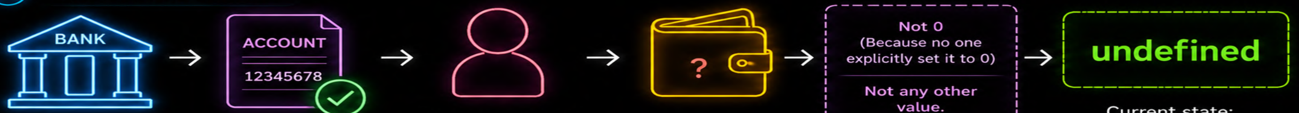
OUTPUT

```
undefined
```

ENGINE THINKING

Variable Created ?	YES
Value Assigned ?	NO
Result ?	undefined

6 REAL WORLD STORY



Bank creates an account. → Account number is created. → Customer has not deposited any money yet. → Balance? Not known. → Not 0 (Because no one explicitly set it to 0) Not any other value. → **undefined**

Current state:
JavaScript represents this as **undefined**.

7 MOST IMPORTANT DISCOVERY

Many beginners think:

`undefined`

means

Nothing exists ✗

Reality:

Variable exists
But value is not assigned yet ✓

8 COMPARE

TYPE	EXAMPLE	MEANING
Undefined	`let account;`	Variable exists Value not assigned
Number	`let account = 0;`	Variable exists Value = 0
String	`let account = "";`	Variable exists Value = Empty String
Boolean	`let account = false;`	Variable exists Value = false

9 TYPE CHECK

```
let account;
console.log(typeof account);
```

typeof

OUTPUT

```
"undefined"
```

10 FINAL FIRST-PRINCIPLES DEFINITION

> Undefined is JavaScript's way of saying:
>
> "The variable exists,
> but no value has been assigned yet."

MENTAL MODEL

Question: Does the variable exist? ? → Answer: YES ✓ → Question: Does it have a value? ? → Answer: NO ✗ → Result: undefined

JS NULL

DATA TYPE IN JAVASCRIPT

05

1 WHAT IS NULL?

null is a special value in JavaScript.
It represents an **intentional** absence of any value.

UNDEFINED		NULL
Variable declared but value not assigned yet.	VS	Variable declared and value intentionally yet to no value.
`undefined`		`null`

undefined means "no value assigned yet".
null means "no value on purpose".

2 JAVASCRIPT ENGINE'S PERSPECTIVE

Engine sees the code:

```
let account = null;
```

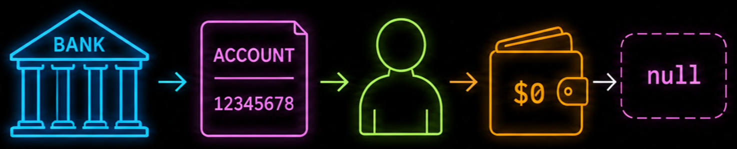
Engine thinks:

The variable exists and the user has intentionally set its value to null (no value).

So engine stores:

`null`
(Intentional no value)

3 REAL WORLD ANALOGY



Bank creates an account. → Account number is created. → Customer had some money. → Now customer withdraws all money. → Balance is set to null because now there is no value.

4 WHEN DO WE USE NULL?

- ✓ When we want to intentionally empty a variable.
- ✓ When a value is not available (but it is expected to be available in future).
- ✓ When we want to reset a variable.
- ✓ In objects, when a property should have no value.

★ We use null when we know the variable exists, but we do not want it to hold any value.

5 MEMORY VISUALIZATION

```
let account = null;
```

Conceptually in memory:

account → null
(Intentional no value)

6 TYPE CHECK

CODE

```
let account = null;
console.log(typeof account);
```

OUTPUT

```
"object"
```

In JavaScript, typeof null is "object".
This is a historical bug in JavaScript.
Do not worry, just remember it.

7 COMPARE ALL 5 PRIMITIVE TYPES

TYPE	EXAMPLE	MEANING
Number	`let a = 10;`	Mathematical value
String	`let a = "Hello";`	Text / Characters
Boolean	`let a = true;`	True or False
Undefined	`let a;`	Declared but value not assigned yet
Null	`let a = null;`	Intentionally set to no value

8 NULL vs UNDEFINED

UNDEFINED

- Variable declared but not assigned.
- Value is not provided yet.
- It happens automatically.
- Represents "not assigned".

VS

NULL

- Variable declared and set to null.
- Value is set intentionally.
- It is done by the developer.
- Represents "no value".

9 LIVE PREVIEW

CODE

```
let name = null;
console.log(name);
console.log(typeof name);
```

OUTPUT

```
null
"object"
```

EXPLANATION

→ Value is null.

→ typeof null returns "object".

10 WHEN DO WE USE NULL?

- ✓ When we want to intentionally empty a variable.
- ✓ When a value is not available (but it is expected to be available in future).
- ✓ When we want to reset a variable.
- ✓ In objects, when a property should have no value.

★ We use null when we know the variable exists, but we do not want it to hold any value.

★ FINAL FIRST-PRINCIPLES DEFINITION

Null is JavaScript's way of saying: "The variable exists, but it is intentionally set to no value."

JS

BIGINT

6th DATA TYPE IN JAVASCRIPT

06

1 THE PROBLEM

Can **Number** store every possible mathematical value?

`let money = 100;`	✓ Works
`let population = 800000000;`	✓ Works
`let stars = 1000000000000;`	✓ Works
9007199254740991	✓ Still safe
9007199254740992	✓ Still looks okay
9007199254740993	⚠ Problem begins!

JavaScript may lose precision.

NUMBER
↓
Safe Integers
↓
9007199254740991
(Number.MAX_SAFE_INTEGER)
↓
Beyond This
↓
Precision Problems
↓
Need New Data Type
↓
BIGINT

2 WHY NUMBER FAILS?

Example:

```
console.log(9007199254740992);
console.log(9007199254740993);
```

Sometimes engine cannot distinguish them accurately.

Because internally **Number** uses:

IEEE 754
Double Precision

which has limited integer precision.

IEEE 754 Double Precision has limited integer precision, so very large numbers cannot be stored accurately.

NORMAL NUMBERS

10
100
1000
1000000
↓
NO PROBLEM

VERY LARGE NUMBERS

99999999999999999999999999999
...
↓
NUMBER CANNOT STORE PERFECTLY
↓
PRECISION LOSS

3 BIGINT IS BORN

JavaScript says:

If **Number** is not enough, create a new type for huge integers.

That type is:

BigInt

4 HOW TO CREATE BIGINT?

Add `n` at the end.

Example:

`123n`

`99999999999999999999999999999n`

`123456789123456789123456789n`

123
↓
NUMBER

123n
↓
BIGINT

BigInt allows JavaScript to store arbitrarily large whole numbers without losing precision! ♥

5 TYPE CHECK

```
console.log(typeof 123n);
```

Output:

```
"bigint"
```

BigInt has its own type: `"bigint"`

6 REAL WORLD USES

Population Database

123456789123456789123456789n

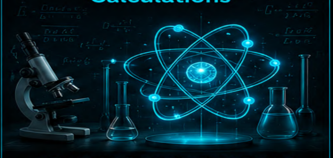
Used to store very large population counts in global systems.

Blockchain (Transactions)

99999999999999999999999999999n

Used in blockchain for storing large transaction IDs, timestamps, hashes.

Scientific Calculations



876543219876543219 87654321n

Used in simulations and research where extremely large integers are required.

Banking Systems

BANK
BANK
1234 5678 9012 3456
CARDHOLDER

99999999999999999999999999999n

Used in core banking to store huge account balances, ledgers, and transaction records.

7 BIGINT OPERATIONS

`10n + 5n`	→	`15n`	Addition
`20n - 10n`	→	`10n`	Subtraction
`10n * 10n`	→	`100n`	Multiplication
`20n / 4n`	→	`5n`	Division (returns BigInt)
`20n % 3n`	→	`2n`	Modulus

8 IMPORTANT RULE

☒ **NOT ALLOWED**

`10n` (BigInt) + `5` (Number) → ERROR ✗

Because BigInt and Number are different types.

✓ **CORRECT**

`10n` (BigInt) + `5n` (BigInt) → `15n`

`BigInt(10)` (BigInt) + `5n` (BigInt) → `15n`

9 MEMORY MENTAL MODEL

```
let money = 99999999999999999999999999n;
```

Conceptually in memory:

money (variable name) → 99999999999999999999999999n (value)

BigInt values are stored as whole numbers, no decimals, with unlimited precision.

Type: `BigInt`

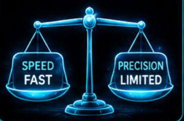
10 NUMBER vs BIGINT

NUMBER

Can store:
- Integers + Decimals

Examples:
- `10`
- `10.5`
- `3.14159`

Limit:
Precision loss for huge integers

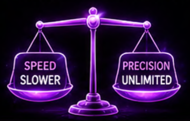


BIGINT

Can store:
- Huge Integers

Examples:
- `10n`
- `99999999999999999999n`
- `123456789123456789n`

Limit:
- No decimals
- Cannot mix with Number



★ Choose Number for normal math.
Choose BigInt for huge whole numbers.

11 VERY IMPORTANT DISCOVERY

NUMBER can store:	BIGINT can store:
✓ `10`	✓ `10n`
✓ `10.5`	✓ `99999999999999999999n`
✓ `3.14159`	✓ `123456789123456789n`

BUT NOT:

☒ `10.5n` (Invalid)

Because BigInt = Big Integer
Only whole numbers

BigInt is perfect for storing very large whole numbers without losing precision.